案例1：nanoGPT

【案例简介】

目前市面上大多数可用的GPT都非常的庞大，初学者很难学习和复现。nanoGPT项目[1]是使用Pytorch对GPT的一个复现，包括训练和推理，其目的是做到小巧、干净、可解释性并且能用于教育。如果市面上的GPT模型是一艘“航空母舰”的话，nanoGPT可看做是一艘游艇，“麻雀虽小，五张俱全”，对于初学者的入门学习有重要的意义。

本案例将训练两个模型：一个是使用由58000首诗词构成的诗歌数据集，训练一个歌词生成的GPT；另一个是使用约124万个字符构成的《天龙八部》文本，训练一个具有《天龙八部》风格的GPT。[1] https://github.com/karpathy/nanoGPT.git

【案例准备】

1. **下载项目源码**（nanoGPT.zip），并解压

（推荐）有条件的可以直接去项目地址下载:git clone https://github.com/karpathy/nanoGPT.git

1. **下载训练数据：**

(1) tang_poet.txt：诗词约有5.8万首

(2) tianlong.txt：《天龙八部》文本，约124万个字符

2. **项目依赖：**

(1) pytorch < 3.0

(2) numpy < 3.0

(3) transformers < 3.0

(4) datasets < 3.0 ： 加载训练数据集

(5) tiktoken < 3.0 ： 可以使用BPE编码

(6) tqdm < 3.0 ： 支持代码进度条

【案例目的】

1. 掌握如何通过pytorch构建一个nanoGPT模型网络：包括模型架构和实现原理，理解其核心组件和功能
2. 学会如何使用nanoGPT进行模型训练，包括数据准备和数据预处理、模型配置、训练过程以及优化技巧
3. 掌握模型的推理过程和细节

【案例实现】

1、诗词生成器

[第一步]训练数据预处理

（1）在项目**nanoGPT/data**文件夹下创建一个**poemtext**文件夹，并将下载好的tang_poet.txt放置在该文件夹下。

（2）在**poemtext**文件夹下创建一个**prepare.py**程序文件，该文件用于对tang_poet.txt文件进行训练数据预处理。

（3）训练数据预处理实现：

① 加载所需要的工具包：

```
import os
import requests
import tiktoken
import numpy as np
```

② 打开数据文件tang_poet.txt

```
input_file_path = 'tang_poet.txt'
with open(input_file_path, 'r') as f:
    data = f.read()
```

③ 按照9:1的比例将数据划分为训练集和测试集

```
n = len(data)
train_data = data[:int(n*0.9)]
val_data = data[int(n*0.9):]
```

④ 使用GPT-2BPE的编码方式对数据进行编码分词，根据输出可知道，训练集有858,024个tokens，测试集有95,229个tokens

```
enc = tiktoken.get_encoding("gpt2")
train_ids = enc.encode_ordinary(train_data)
val_ids = enc.encode_ordinary(val_data)
print(f"train has {len(train_ids):,} tokens")
print(f"val has {len(val_ids):,} tokens")
```

⑤ 将训练集和测试集另存为train.bin和val.bin二进制文件，用于训练使用

```
train_ids = np.array(train_ids, dtype=np.uint16)
val_ids = np.array(val_ids, dtype=np.uint16)
train_ids.tofile(os.path.join(os.path.dirname(__file__),'train.bin'))
val_ids.tofile(os.path.join(os.path.dirname(__file__),'val.bin'))
```

运行完prepare.py文件后，将会在poemtext文件夹下生成train.bin和val.bin文件，该文件将用于模型的训练。

[第二步]模型的训练

数据准备好后，模型的训练非常简单：

① 进入nanoGPT/config文件夹，并创建train_poemtext_char.py文件

② 然后将一下内容复制进文件：

```
out_dir = 'out-poemtext-char'
eval_interval = 250 # keep frequent because we'll overfit
eval_iters = 200
log_interval = 10 # don't print too too often

always_save_checkpoint = False

dataset = 'poemtext'
gradient_accumulation_steps = 1
batch_size = 64
block_size = 256 # context of up to 256 previous characters

n_layer = 6
n_head = 6
n_embd = 384
dropout = 0.2

learning_rate = 1e-3 # with baby networks can afford to go a bit higher
max_iters = 5000
lr_decay_iters = 5000 # make equal to max_iters usually
min_lr = 1e-4 # learning_rate / 10 usually
beta2 = 0.99 # make a bit bigger because number of tokens per iter is small

warmup_iters = 100 # not super necessary potentially
```

其中我们需要重点关注的是out_dir和dataset参数，out_dir即模型文件的保存目录，dataset是训练集train.bin和测试机val.bin所在的文件夹。

至于其他参数，本次案例不做详细讲解，课后可以自由进行讨论研究。

③模型训练：

[1] **进入项目**：打开计算机控制台，切换到本项目的环境，并进入到nanoGPT项目所在文件夹：

[2] **训练方式**：

（1）如果你的电脑装有NVIDIA显卡，并支持cuda计算，在控制台下使用如下命令：

```
python train.py config/train_poemtext_char.py
```

（2）如果你的电脑没有NVIDIA显卡，则使用CPU进行训练，在控制台下使用如下命令：

```
python train.py config/train_poemtext_char.py --device=cpu --compile=False --eval_iters=20 --log_interval=1 --block_size=64 --batch_size=12 --n_layer=4 --n_head=4 --n_embd=128 --max_iters=2000 --lr_decay_iters=2000 --dropout=0.0
```

[3] **训练过程**：执行以上命令后，如果控制台上显示iter、loss、time等信息，说明模型已经开始训练，直到训练结束。

模型训练结束后，会在项目目录下生成一个文件权重out-poemtext-chat/ckpt.pt，该模型权重就是由58000首诗词tang_poet.txt数据得到的GPT。

[第三步]模型的推理及采样：

现在我们已经训练好一个GPT模型了！那么现在利用该模型来生成一些内容：

（1）如果你的电脑装有NVIDIA显卡，并支持cuda计算，可以使用显卡进行推理，在控制台下使用如下命令：

```
python sample.py --out_dir=out-poemtext-char
```

（2）如果你的电脑没有NVIDIA显卡，则使用CPU进行推理，在控制台下使用如下命令：

```
python sample.py --out_dir=out-poemtext-char --device=cpu
```

运行以上代码后，控制台将出现诗词GPT采样结果

【思考】

1. 使用《天龙八部》tianlong.txt数据集训练一个GPT模型，并生成内容看看效果。
2. 思考config/train_poemtext_char.py训练文件中其他参数的含义，修改其参数并重新进行训练，看看其效果怎么样。
3. 研究模型采样文件sample.py，思考模型的推理过程是怎么样的?

案例2：开源大语言模型Qwen1.5的使用

【案例简介】

作为开源大语言模型学习案例，考虑到不同学习者的硬件条件不同，决定使用Qwen1.5-0.5B模型权重作为学习模型，其参数量仅有5亿个参数，能在普通的个人电脑上运行。通过本案，可以学习到如何加载和使用一个大语言模型，如何使用大语言模型构建一个对话应用，以及了解一个ChatBot应用的构建逻辑等内容。

【案例准备】

1. 下载项目源码（Qwen1.5.zip），并解压

（推荐）
有条件的可以直接去项目地址下载: （该项目已更新为Qwen2.5，可能会git不到相关的版本，建议手动下载）

git clone https://github.com/QwenLM/Qwen1.5.git

2. 下载Qwen1.5-0.5B模型权重文件：

（1）（不推荐）手动进行下载

打开https://www.modelscope.cn/models/qwen/Qwen1.5-0.5B-Chat/files，下载所有的文件，并保存到你的本地目录Qwen1.5-0.5B-Chat；

（2）**（推荐）**使用modelscope进行下载

Modelscope魔搭社区[2]是一个由阿里巴巴达摩院推出的开源模型平台，旨在打造成为下一代的模型即服务（MaaS）共享平台。是类似于国外huggingface社区，旨在降低AI应用的门槛，使模型的应用变得更加简单，提供一个包含各领域先进机器学习模型的开源社区，促进模型的共享与复用，减少重复研发成本，推动AI技术的普及和绿色“数字经济”的发展。[2] 魔搭社区：https://modelscope.cn/home

如果在环境中已经安装了modelscope工具包，运行一下命令即可下载Qwen1.5-0.5B模型权重文件：

或者pip install modelscope 在环境中安装

```
1. from modelscope import snapshot_download
2. model_dir = snapshot_download('qwen/Qwen1.5-0.5B-Chat',cache_dir='./')#cache_dir='./'为指定下载目录，不然会下载到C盘的缓存中
```

3.项目依赖：

创建新的虚拟环境（是一个比较老的案例，请严格按照该环境配置，不然可能会出现环境包的冲突）

（1）python >= 3.8 （conda create -n Qwen1.5 python=3.8）

（2）pytorch >= 1.12，推荐使用2.0及以上的版本(cpu版本为例：pip install torch torchvision torchaudio)

（3）transformers >= 4.32（pip install transformers)

（4）CUDA >= 11.4 (GPU环境会的同学自动部署，以cpu为例的同学可以不用理会)

（5） accelerate >= 0.26.0（pip install accelerate）

【案例目的】

1. 如何使用开源大语言模型：例如如何加载模型权重，tokenizer，推理
2. 如何使用大语言模型进行构建一个应用
3. 了解一个ChatBot应用的构建逻辑

【案例实现】

1、Qwen1.5-0.5B的加载以及推理

代码实现：

① 加载所需要的工具包：

```
1. from transformers import AutoModelForCausalLM, AutoTokenizer
2. import torch
3. device = 'cuda' if torch.cuda.is_available() else 'cpu'
4. model_path = './qwen/Qwen1.5-0.5B-Chat'
```

device：如果支持GPU，则使用cuda推理，否则使用cpu进行推理

model_path：模型路径（这一步需要修改为自己下载模型的实际路径）

② 加载模型参数和tokenizer

```
model = AutoModelForCausalLM.from_pretrained(
    model_path,
    torch_dtype="auto",
    device_map="auto"
)
tokenizer = AutoTokenizer.from_pretrained(model_path)
```

③ 定义提示语（prompt）

```
prompt = "简单介绍一下你自己"
messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": prompt}
]
```

④ 将提示语messages转换为一个格式化的对话字符串text，并将text转化为张量格式

```
text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True
)
model_inputs = tokenizer([text], return_tensors="pt").to(device)
print(text)
```

text的输出结果为：

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
简单介绍一下你自己<|im_end|>
<|im_start|>assistant
```

⑤ 使用generate()函数生成内容，并保留生成的内容

```
generated_ids = model.generate(
    model_inputs.input_ids,
    max_new_tokens=512
)
generated_ids = [
    output_ids[len(input_ids):] for input_ids, output_ids in zip(model_inputs.input_ids, generated_ids)
]
```

⑥ 使用tokenizer.batch_decode()对生成的ids进行解码，得到实际内容

```
response = tokenizer.batch_decode(generated_ids, skip_special_tokens=True)[0]
print(response)
```

response实际内容为：

```
我是一个人工智能助手，可以帮助您解答问题、提供信息和执行任务。我可以与您聊天，搜索互联网上的信息，还可以帮助您管理日程安排等等。如果您有任何问题，请随时告诉我，我会尽力为您提供帮助。
```

2、使用Qwen1.5-0.5B构建一个ChatBot

（1）进入Qwen1.5项目文件夹，打开Qwen1.5/examples/demo/web_demo.py文件，环境需要安装gradio（pip install gradio）

（2）找到15行代码:

DEFAULT_CKPT_PATH = 'Qwen/Qwen1.5-7B-Chat'

将其值修改为Qwen1.5-0.5B模型的实际路径

例如:

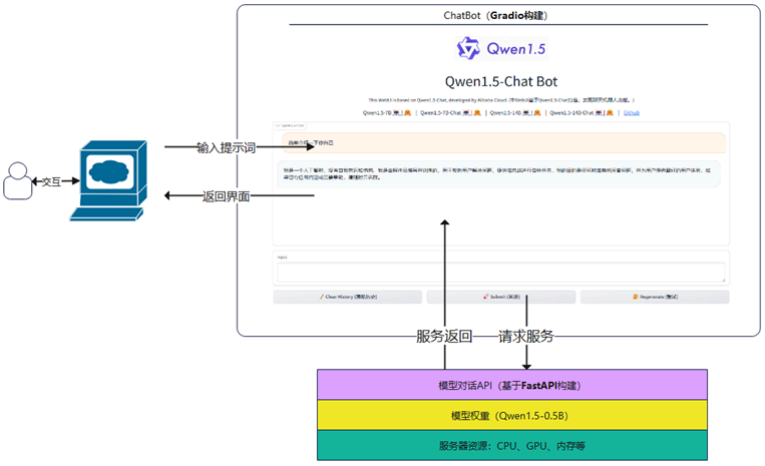
```
DEFAULT_CKPT_PATH = D:\python\Qwen1.5-main\qwen\Qwen1.5-0.5B-Chat'
```

（3）接着直接运行web_demo.py文件：python web_demo.py

然后再浏览器中输入:localhost:8000，就可以在对话框中进行对话，如图：

ChatBot的构建思路

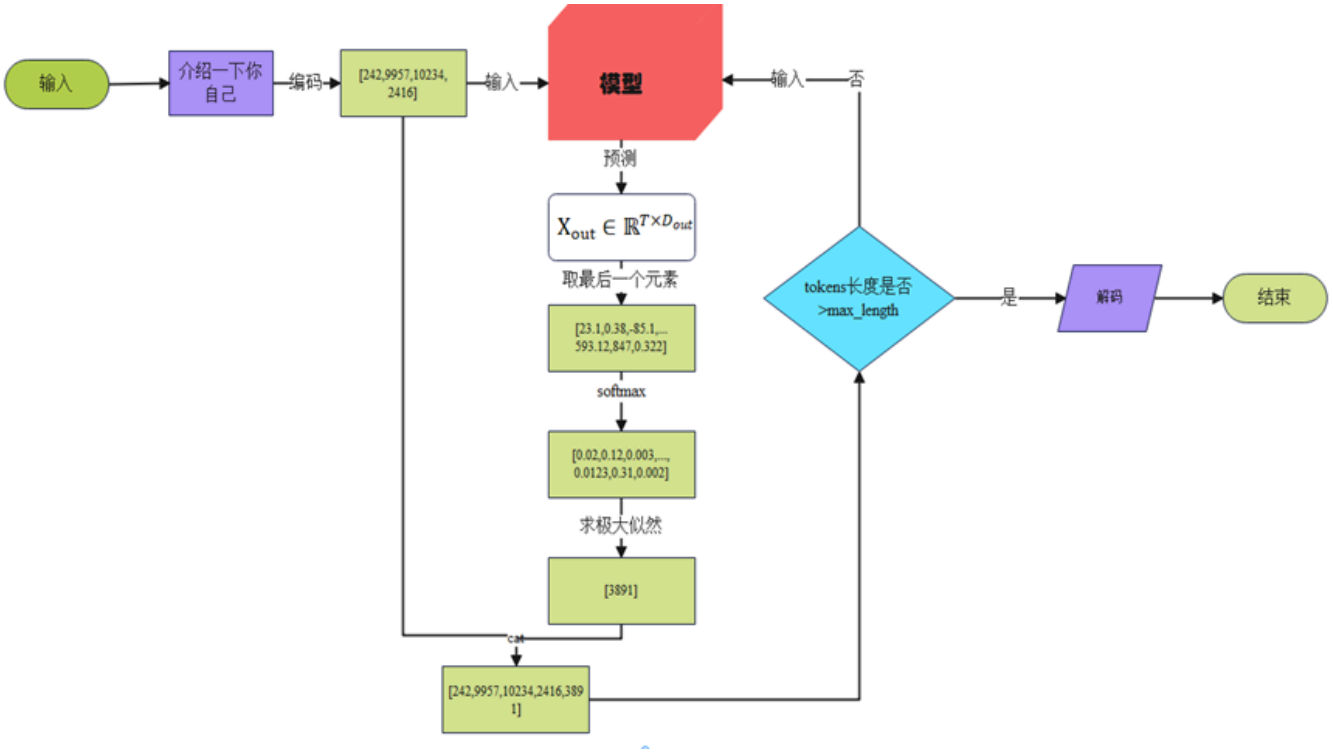
上述案例中ChatBot的构建主要由模型权重服务、API服务和Web服务构成。其中API主要由**FastAPI框架**实现，该框架可快速的构建API，ChatBot就是对案例2中的对话过程服务进行封装成API，可供外部程序进行调用；Web服务由**Gradio框架**实现，是一个用于构建机器学习模型交互界面的Python库，它极大地简化了创建模型演示和用户界面的过程。Gradio允许开发者仅通过几行代码就能将复杂的机器学习模型转化为用户友好的Web应用，而不需要深厚的前端开发知识。ChatBot中，通过使用Gradio完成页面的设计与交互按钮，通过调用FastAPI提供的API，完成与用户对话。下图是用户使用ChatBot的交互逻辑图。



用户通过浏览器访问到`ChatBot`的`web`页面，然后在提示词框中输入“提示词”，点击`Submit`进行发送给对话`API`，接着将“提示词”传送给模型进行预测，然后将预测得到的结果返回后端，再将内容展示到前端页面。

以上就是``ChatBot``构建的基本逻辑。如果想要更好的构建一个``ChatBot``应用，学习者可以到``FastAIP``（https://fastapi.tiangolo.com/）和``Gradio``的官方网站（https://gradio.app/ ）进行学习，本案例将不再讨论其细节。

案例3：大语言模型的推理过程及细节（选学）



首先会有一个提示词“介绍一下你自己”，接着使用``tokenizer``对提示词进行编码，得到类似[242,9957,10234,2415]的向量，接着将输入向量传入模型进行预测推理，会得到一个$X_{out} \in \mathbb{R}^{T \times D_{out}}$的结果矩阵，根据``Transformers``的训练过程可知，$X_{out}$中的最后一个向量才是本次的预测结果，所以取最后一个元素，其结果为[23.1,0.38,-85.1,...,593.12,847,0.322]`，其长度为词表长度，接着使用`Softmax`函数将结果向量转化为概率，再使用极大似然估计函数获得概率最大的值所对应的下标值，其值就是本次推理的结果`[3891]`。```

由于``Transformers``是一个自回归模型，需要将原始输入向量 [242,9957,10234,2415] 和本次推理结果向量 [3891] 进行拼接，得到 [242,9957,10234,2415,3891]，接着进行判断，目前``tokens``结果长度是否超过最大长度``max_length``，如果未超过，则重复以上过程；否则使用``tokenizer``对最后的结果向量 [242,9957,10234,2415,3891]`进行解码，解码后的内容就是本次提示词的回答内容。

【案例代码】开源大语言模型的推理过程

① 导入所需要使用的依赖包

```
import os
import json
import torch
import tqdm
import torch.nn.functional as F
from transformers import Qwen2Model, Qwen2Config, Qwen2ForCausalLM, Qwen2Tokenizer
from safetensors.torch import load_file

# 检查是否有GPU可用
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

② 使用safetensor的形式，手动加载模型参数

```
model_root = 'D:/python/Qwen1.5-main/qwen/Qwen1.5-0.5B-Chat/'
config = Qwen2Config.from_pretrained(model_root + 'config.json')
model = Qwen2ForCausalLM(config)

# 列出所有 safetensors 文件
filelist = os.listdir(model_root)
safetensors_files = [file for file in filelist if file.endswith('.safetensors')]
safetensors_files.sort()

# 加载模型权重
model_state_dict = {}
for model_file in safetensors_files:
    model_file_path = model_root + model_file
    weights = load_file(model_file_path)
    model_state_dict.update(weights)

# 将模型权重加载到模型中
model.load_state_dict(model_state_dict)
```

③ 如果有多个GPU，可以将模型分布式部署到多个GPU上

```
# 如果有多个GPU，使用 DataParallel
if torch.cuda.device_count() > 1:
    print(f"Using {torch.cuda.device_count()} GPUs")
    model = torch.nn.DataParallel(model)

# 将模型移动到GPU上
model.to(device)
```

④ 手动加载tokenizer，并手动加载特殊token

```
# 定义词汇表和分词规则的文件路径
vocab_file = model_root + 'vocab.json'  # 例如 'vocab.json'
merges_file = model_root + 'merges.txt'  # 例如 'merges.txt'
tokenizer_file = model_root + 'tokenizer.json'  # 例如 'merges.txt'
tokenizer_config = model_root + 'tokenizer_config.json'  # 例如 'tokenizer_config'
# 加载词汇表和分词规则
tokenizer = Qwen2Tokenizer(vocab_file=vocab_file,
                    merges_file=merges_file,
                    tokenizer_file = tokenizer_file,
                    tokenizer_config = tokenizer_config)
# 添加新的token
new_tokens = [
    {"content": "<|im_start|>", "lstrip": False, "normalized": False, "rstrip": False, "single_word": False, "special": True, "id": 151644},
    {"content": "<|im_end|>", "lstrip": False, "normalized": False, "rstrip": False, "single_word": False, "special": True, "id": 151645}
]

# 将新token添加到tokenizer
tokenizer.add_special_tokens({'additional_special_tokens': [token['content'] for token in new_tokens]})
```

⑤ 初始化提示词，并设置最大生成的最大token长度max_length

```
# 初始化输入
input_text = '简单介绍一下成都东软学院'
inputs = tokenizer(input_text, return_tensors="pt")
input_ids = inputs['input_ids'].to(device)

# 设置生成的最大长度
max_length = 40
```

⑥ 生成内容

```
# 开始生成
model.eval()
with torch.no_grad():
    for _ in range(max_length - input_ids.shape[1]):
        outputs = model(input_ids)
        logits = outputs.logits
        next_token_logits = logits[:, -1, :]
        next_token_probs = F.softmax(next_token_logits, dim=-1)
        next_token_id = torch.argmax(next_token_probs, dim=-1).unsqueeze(-1)
        input_ids = torch.cat([input_ids, next_token_id], dim=-1)
        print(f"{tokenizer.decode(next_token_id[0])}", end='')
        if tokenizer.decode(next_token_id[0]).strip() in ['.', '!', '?']:
            break

generated_text = tokenizer.decode(input_ids[0], skip_special_tokens=True)

```

```
print(generated_text)
```

输出结果：

```
<|im_start|>成都东软学院是一所位于四川省成都市的全日制普通本科高校，成立于2005年，是一所以工为主，理、文、
简单介绍一下成都东软学院 成都东软学院是一所位于四川省成都市的全日制普通本科高校，成立于2005年，是一所以工为
主，理、文、
```

Qwen2.5版本

```
import torch
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    TextStreamer
)

# # 设备配置（支持多GPU自动分配）
# device = "cuda" if torch.cuda.is_available() else "cpu"
# torch_dtype = torch.bfloat16 if torch.cuda.is_bf16_supported() else torch.float16

# 强制指定CPU运行（移除GPU相关判断）
device = "cpu"  # 强制指定设备为CPU
torch_dtype = torch.float32
# 模型加载
model_path = "D:/python/Qwen2.5-main/qwen/Qwen2.5-0.5B-Instruct"
model = AutoModelForCausalLM.from_pretrained(
    model_path,
    # device_map="auto",
    torch_dtype=torch_dtype,
    trust_remote_code=True
).to(device).eval()

# 分词器配置
tokenizer = AutoTokenizer.from_pretrained(
    model_path,
    padding_side='left',
    trust_remote_code=True
)
special_tokens = {
    "additional_special_tokens": ["<|im_start|>", "<|im_end|>"],
    "pad_token": "<|extra_1|>"
}
tokenizer.add_special_tokens(special_tokens)
model.resize_token_embeddings(len(tokenizer))

# 流式输出配置
streamer = TextStreamer(
    tokenizer,
    skip_prompt=True,
    skip_special_tokens=True
)
```

```python
# 对话模板构建
messages = [
    {"role": "system", "content": "你是有帮助的AI助手"},
    {"role": "user", "content": "简单介绍一下成都东软学院"}
]
prompt = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True
)

# 生成参数设置
generation_config = {
    "max_new_tokens": 400,
    "temperature": 0.7,
    "top_p": 0.9,
    "repetition_penalty": 1.1,
    "do_sample": True,
    "eos_token_id": tokenizer.encode("<|im_end|>")[0],
    "streamer": streamer
}

# 执行生成
input_ids = tokenizer(
    [prompt],
    return_tensors="pt"
).to(model.device)

print("生成结果：")
try:
    _ = model.generate(
        **input_ids,
        **generation_config
    )
    print("\n生成完成。")
except Exception as e:
    print(f"生成过程中出现错误：{e}")
```

```
生成结果：
成都东软学院是经四川省人民政府批准成立，由四川大学和成都信息工程学院共同投资举办的一所公办全日制普通本科高校。学校位于四川省成都市，是全国首批试点建设的“双一流”建设高校之一，也是西部地区唯一一所具有硕士、博士学位授予权的独立学院。

学校设有7个二级学院：机械工程学院、计算机科学与技术学院、经济管理学院、国际商学院、土木工程学院、艺术与传媒学院、外国语学院；2013年被授予“教育部卓越工程师教育培养计划试点项目院校”，2014年被认定为国家级大学生校外实践教育基地。此外，学校还拥有中国工程院院士张杰院士领衔的工程技术研究中心，并设有研究生院、继续教育学院等多处教学科研机构。

生成完成。
```

由于模型较小，生成出的结果和实际情况相比差异较大